Team Name : **Alonecoder**

Team Leader Name : **Karthikeyan M**

Problem Statement :

From a 100,000-candidate pool, rank the top 100 for a Senior AI Engineer role by understanding who actually fits, not by matching keywords.

Solution Overview

- What is your proposed solution?
- What differentiates your approach from traditional candidate matching systems?
  - Proposed solution: a hybrid ranker with three lenses. Semantic embeddings read each candidate's career story, interpretable recruiter rules judge the real role and production evidence, and behavioral signals plus trap/honeypot guards refine the result.
  - What sets it apart: we proved the skills list is noise (every skill appears about 12,000 times, sprayed across all profiles), so we ignore it and read the career narrative instead. We also model the JD's anti-patterns (research-only, computer vision, consulting, title-chasers) as negative signals, which keyword matchers cannot do.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
  - Key requirements: production embeddings and retrieval, vector DB or hybrid search, ranking evaluation (NDCG, MRR, MAP), strong Python, 5 to 9 years at product (not services) companies, Pune or Noida, active on the platform. Disqualifiers: pure research, recent LangChain-only work, title-chasers, consulting-only careers, CV/speech without NLP or IR.
  - Signals that matter most: current and past job title, career-history free text showing real production retrieval and ranking work, product vs services trajectory and tenure, and behavioral availability. Beyond keywords we use semantic match of the career story plus role classification and negative-facet penalties.

Ranking Methodology

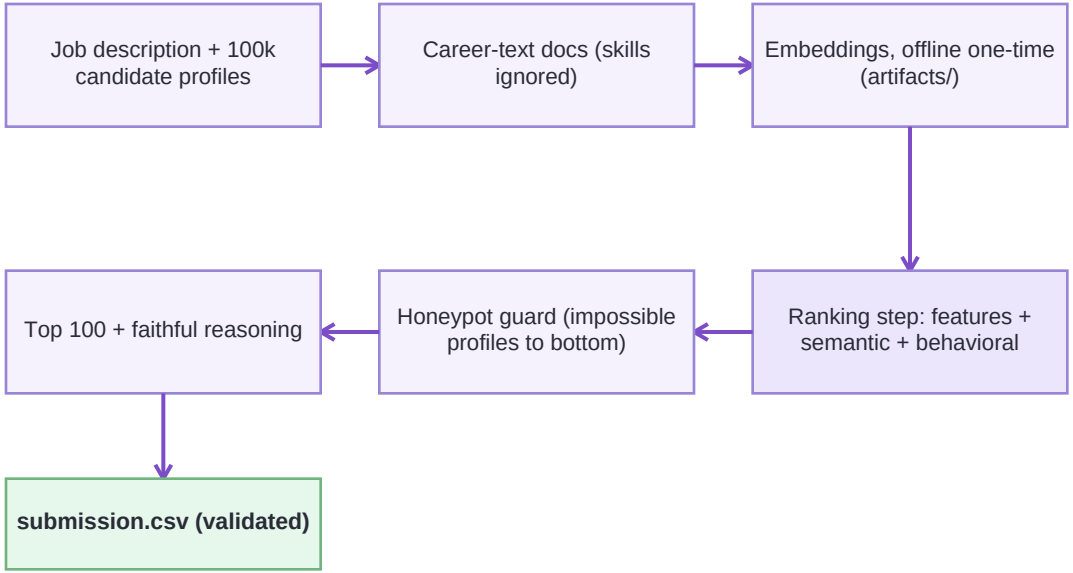
- How does your system retrieve, score, and rank candidates?
- What models, algorithms, or heuristics are used?
- How are multiple candidate signals combined into a final ranking?
  - Retrieve, score, rank: embed each profile's career text, score it against the JD split into positive and negative facets, add interpretable feature scores, multiply by an availability modifier, then sort and take the top 100.
  - Models and heuristics: a CPU sentence-transformer (all-MiniLM-L6-v2) for cosine similarity, a rule-based role classifier, regex evidence extraction, a gaussian experience-band, and strict honeypot checks.
  - Combining signals: weighted sum (role 0.34, semantic 0.30, evidence 0.22, band 0.08, location 0.06) times a behavioral multiplier from 0.55 to 1.08. Honeypots are forced to zero.

Explainability & Data Validation

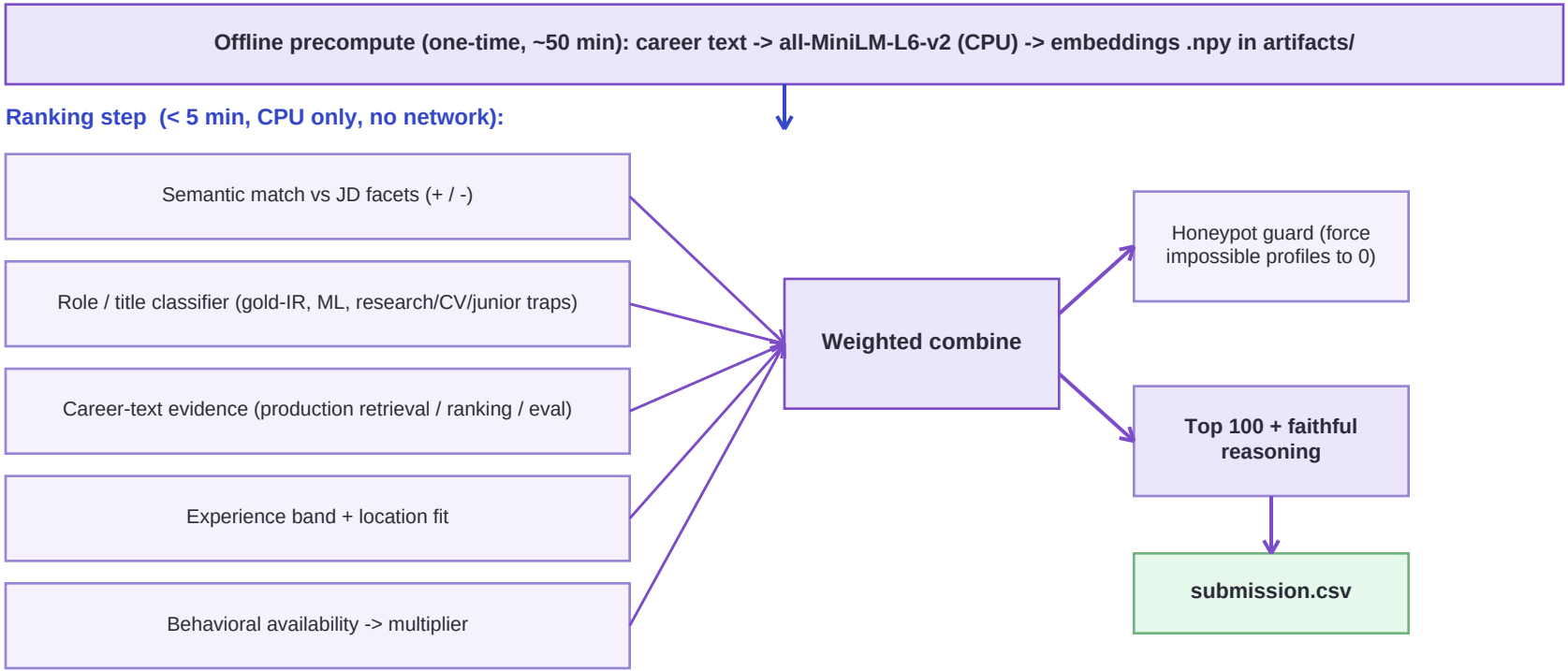
- How are ranking decisions explained?
- How do you prevent hallucinations or unsupported justifications?
- How does your solution handle inconsistent, low-quality, or suspicious profiles?
  - How decisions are explained: every candidate gets a one to two sentence reason citing real facts, the job title, company, years of experience, the evidence found, and key signal values.
  - Preventing hallucination: reasons are generated programmatically from the exact features that drove the score, so nothing is invented and each one names an honest concern.
  - Suspicious profiles: a strict impossibility guard (job duration longer than time since it started, many expert skills with 0 months used, end-before-start dates) forces the roughly 80 honeypots to the bottom. Zero reached our top 100.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

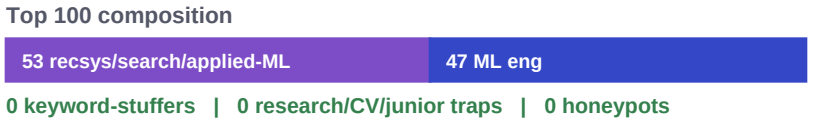


System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?
- How does your solution meet the challenge's runtime and compute constraints?
  - Ranking quality: the top 100 are 53 recommendation, search and applied-ML engineers plus 47 ML engineers. Zero keyword-stuffers, zero research/CV/junior traps, zero honeypots. The top 10 all show production retrieval and evaluation evidence, 5 to 8 years, in Indian tech hubs.
  - Constraints: the ranking step runs in about 106 seconds (limit 5 minutes), under 16 GB RAM, CPU only, with no network and no LLM calls. The heavy embedding is a one-time offline precompute.



Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?
  - Python 3.11 core. sentence-transformers (all-MiniLM-L6-v2) for small, fast, GPU-free embeddings. NumPy for fast vector math in the timed step. uv for reproducible environments. Streamlit for the hosted demo. PyMuPDF for this deck.
  - Why these: every choice respects the CPU, 5-minute, no-network budget, keeps the system fully reproducible offline, and stays transparent enough to defend in the final interview.

Submission Assets

Github video etc

- GitHub repository: github.com/karywnl/fitsignal (full source, README, single reproduce command).
- Live demo: huggingface.co/spaces/karywnl/fitsignal (Streamlit sandbox).
- Ranked output: Alonecoder.csv (top 100 candidates, validated).
- This deck (PDF) and submission_metadata.yaml (team, compute, AI-tools declaration).

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU